

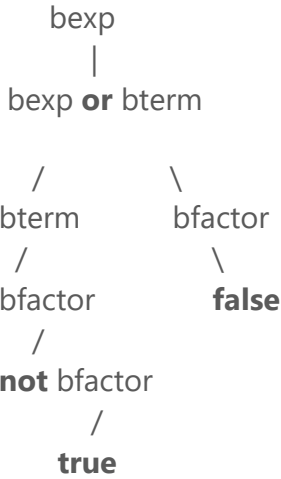
Tema1:

```
bexp    →    bexp or bterm | bterm

bterm   →    bterm and bfactor | bfactor

bfactor →    not bfactor | (bexp) | true | false
```

1.1- Construir el árbol de análisis sintáctico para la cadena de entrada: **not true or false**) (5 pts)



HAY ERROR SINTACTICO

1.2- ¿Existe un error léxico en el proceso? Justificar (5 pts)

no, todos los símbolos pertenecen al lenguaje, tokens válidos.

TEMA 2 (15 puntos) Expresiones Regulares y Autómatas:

Considere el lenguaje reducido math que acepta expresiones aritméticas incluyendo números sin signo (enteros o con punto flotante), operadores aritméticos (+, -, * y /), paréntesis abierto y cerrado, variables (inician con una letra y pueden venir seguido de letras o dígitos), y el operador de asignación (=). Un ejemplo de fuente valido podría ser:

Ejemplo.math

y = 3*x + 10
resultado = (y - 100) / 2

- a) **Proponga una expresión regular R para reconocer todos los tokens permitidos del lenguaje reducido (unificar todas las expresiones regulares en una sola).** (5 pts)
R: $[0-9](\.[0-9])? | + | - | * | / | (|) | = | ([a-zA-Z][a-zA-Z0-9])^*$
- b) **Grafique un NFA (Autómata Finito No-Determinístico) generado a partir de la expresión regular R utilizando las reglas de Thompson. (5 pts)**

